

Machine Learning With NLP

DATA PREPROCESSING

Step 1 : Importing the required Libraries

- NumPy is a Library which contains Mathematical functions.
`>>import numpy as np`
- Pandas is a Library used to load and manage the data sets.
`>>import pandas as pd`

❖ NumPy Methods

Methods	Uses
array()	Used to create an array
arrange()	Used to arrange the array in step values
shape()	To return the dimensions of array
dtype()	To return the data type of the array
size()	To return the number of array elements
sqrt()	To return the square root of the number
median()	To return the the median of array
a[0:2]	To select items at index of 0 and 1 of array (2 is excluded)
a[: :-1]	To print reversed array

❖ Pandas Methods

Methods	Uses
read_csv()	To read data from the csv file
head()	To display first few rows of the csv files
tail()	To display last few rows of the csv files
info()	Display information about the dataframe such as number of rows, number of columns, null values, data types, etc
describe()	Used to calculate statistical data for numerical columns
drop()	Used to drop the specified rows or columns
iloc[]	Used to select rows and columns of the dataframe by index
duplicated()	To check for the duplicated rows in the dataframe
min()	To find the minimum value of columns in dataframe
max()	To find the maximum value of columns in dataframe





sample()	Display few random rows from the dataframe
value_counts()	Used to count the number of occurrences of each unique value in a column of the dataframe
dropna()	Drop rows with missing values from the dataframe
shape()	Display the number of rows and columns
count()	Calculate number of non null values in each dataframe
fillna()	Used to fill missing values in dataframe
drop_duplicates()	To access the output DataFrame without duplicate rows
mean()	To find the mean
to_csv()	To save the dataframe into a csv file format

Step 2 : Importing the Data Set

- Data sets are generally available in .csv format.
- A csv file stores tubular data in plain text
- Each lines of file is a data record
- Using `read_csv()` method of pandas library we can load the data as a dataframe.
- Then we make the separate matrix and vector of independent and dependent variables from dataframe.

Step 3 : Handling the missing Data:

- The data we get is rarely homogeneous.
- Data can be missing due to various reasons and needs to be handled so that it does not reduce the performance of our machine learning model.
- We can replace the missing data by the Mean or Median of the entire column.
- We use `Imputer` class of `sklearn.preprocessing` for this task.

TRAINING AND TESTING THE MODEL USING SKIKIT-LEARN

Sklearn :

Scikit-learn, often abbreviated as sklearn, is a popular open-source machine learning library for Python. It is widely used by data scientists, researchers, and machine learning practitioners to develop and implement machine learning algorithms. Scikit-learn provides a wide range of tools for various machine learning tasks, including classification, regression, clustering, dimensionality reduction, and more.

Step 1 : Importing the required Libraries

Import the required module for splitting data using scikit-learn.

```
>> from sklearn.model_selection import train_test_split
```

Step 2 :Split Data into Training and Testing Sets:

- a. Use the `train_test_split()` function to split your preprocessed dataset into training and testing subsets.

```
>>xtrain, xtest, ytrain, ytest = train_test_split(features, labels,  
test_size=0.2, random_state=42)
```

- b. Now the data is splits into training and testing.



Linear Regression:

Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to the observed data.

Steps:

- a. Import the required module for linear regression from scikit-learn.

```
>> from sklearn.linear_model import LinearRegression
```

- b. Initialize and Train the Linear Regression Model:

Create an instance of the **LinearRegression** class, fit it to the training data, and make predictions on the test data.

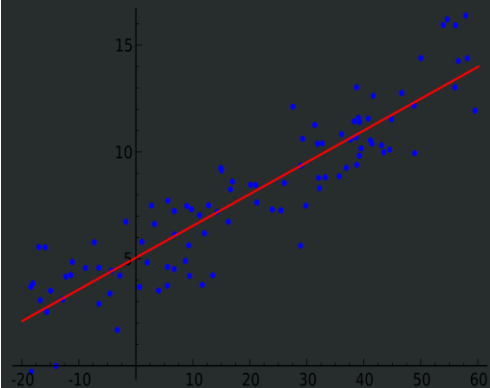
```
>> model = LinearRegression()
>> model.fit(xtrain, ytrain)
>> ypred = model.predict(xtest)
```

- c. Evaluate Model Performance (for Regression):

Depending on the problem, you can use appropriate metrics like Mean Absolute Error (MAE), Mean Squared Error (MSE), or R-squared to evaluate the performance of the regression model.

```
>> from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
```

```
>> mae = mean_absolute_error(ytest, ypred)
>> mse = mean_squared_error(ytest, ypred)
>> r2 = r2_score(ytest, ypred)
```



Logistic Regression:

Logistic Regression is a statistical method used for binary classification tasks, where the goal is to predict one of two possible outcomes based on input features. Despite its name, logistic regression is a classification algorithm, not a regression algorithm like linear regression.

Steps:

a. Import Necessary Modules:

Import the required module for logistic regression from scikit-learn.

```
>> from sklearn.linear_model import LogisticRegression
```

b. Initialize and Train the Logistic Regression Model:

Create an instance of the **LogisticRegression** class, fit it to the training data, and make predictions on the test data.

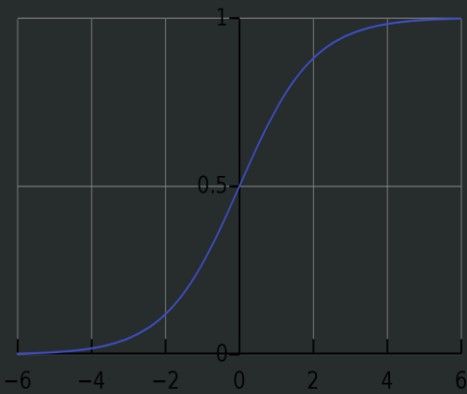
```
>> model = LogisticRegression()
>> model.fit(xtrain, ytrain)
>> ypred = model.predict(xtest)
```

c. Evaluate Model Performance (for Classification):

For classification tasks, you can use metrics like accuracy and confusion matrix to evaluate the performance of the logistic regression model.

```
>> from sklearn.metrics import accuracy_score, confusion_matrix

>> accuracy = accuracy_score(ytest, ypred)
>> cm = confusion_matrix(ytest, ypred)
```





Sci Kit Learn Methods

Methods	Uses
<code>model.fit(x,y)</code>	Fit the model to the data
<code>model.predict(x)</code>	Predict the labels
<code>model.score(x,y)</code>	To find accuracy score
<code>mean_absolute_error(ytrain,ytest)</code>	To find mean absolute error
<code>mean_squared_error(ytrain,ytest)</code>	To find mean squared error
<code>classification_report(ytest,ypred)</code>	To find metrics of precision, recall, F1 score and support
<code>train_test_split(x,y ,test_size=0.2,random state=1)</code>	Splitting data for training and testing the model
<code>a = LabelEncoder()</code>	Encode target labels with value between 0 and (numberOfLabel-1). Use to normalize labels.
<code>a.fit_transform(y)</code>	Fit the label encoder and return encoded labels

NATURAL LANGUAGE PROCESSING USING NLTK

NLTK :

NLTK (Natural Language Toolkit) is a powerful Python library designed to work with human language data. It provides various tools and resources for tasks related to natural language processing (NLP).

1. Text Preprocessing:

NLTK offers functions to help you clean and preprocess text data. This includes tasks like tokenization (breaking text into words or sentences), stemming (reducing words to their root form), and lemmatization (reducing words to their base or dictionary form).

a. Tokenization:

Tokenization involves breaking a text into individual words or sentences (tokens). NLTK provides functions to tokenize text into words and sentences, which is essential for various NLP tasks.



```
>>from nltk.tokenize import word_tokenize, sent_tokenize
```

```
>>text = "This is an example sentence. And another one."
```

```
>>words = word_tokenize(text)
```

```
>>sentences = sent_tokenize(text)
```

b. **Stopword Removal:**

Stopwords are common words like "the," "and," "is," etc., that are often removed from text because they don't carry significant meaning. NLTK provides a list of stopwords and functions to remove them from your text.

```
>> from nltk.corpus import stopwords
```

```
>>stop_words = set(stopwords.words("english"))
```

```
>>filtered_words = [word for word in words if word.lower() not in
stop_words]
```

c. **Stemming and Lemmatization:**

These techniques reduce words to their base or root form to normalize variations of words. NLTK provides stemmers and lemmatizers for this purpose.

```
>> from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```
>>stemmer = PorterStemmer()
```

```
>>lemmatizer = WordNetLemmatizer()
```

```
>>stemmed_words = [stemmer.stem(word) for word in words]
```

```
>>lemmatized_words = [lemmatizer.lemmatize(word) for word in
words]
```

d. **Lowercasing:**

Converting all text to lowercase can help in making text case-insensitive and reducing the vocabulary size.

```
>> lowercase_words = [word.lower() for word in words]
```

2. **POS Tagging** (Parts of speech tagger):

Part-of-speech (POS) tagging is the process of labeling each word in a sentence with its corresponding grammatical part of speech, such as noun, verb, adjective, etc. The Natural Language Toolkit (NLTK) is a popular Python library that provides various tools for working with human language data, including POS tagging.

a. **import libraries:**

```
>> import nltk
```

```
>>nltk.download('punkt') # Download tokenization data
```

```
>>nltk.download('averaged_perceptron_tagger') # Download POS
tagging data
```

- b. Tokenize your input text into words:

```
>> from nltk.tokenize import word_tokenize

>>sentence = "your data"
words = word_tokenize(sentence)
```

- c. Perform POS tagging using NLTK's **pos_tag** function:

```
>> from nltk import pos_tag

>>pos_tags = pos_tag(words)
>>print(pos_tags)
```

- d. The **pos_tag** function returns a list of tuples, where each tuple contains a word and its corresponding POS tag.

❖ NLTK (Natural Language ToolKit)

Methods	Uses
from nltk.corpus import brown brown.words(text_id)	Returns pretokenized document as a list of words
from nltk.tokenize import sent_tokenize, word_tokenize word_tokenize(para) sent_tokenize(para)	word_tokenize() returns the list of words from given para sent_tokenize() returns the list of sentences from given para
Stem and Lemmatizing: Words = input.lower().split(' ') Stem = nltk.PorterStemmer [Stem.stem(t) for t in Words] Lemm = nltk.WordNetLemmatizer() [Lemm.lemmatize(t) for t in Words]	- Lowercase the input data - Creating stemmer - Creating list of stem words - Initializing WordNetLemmatizer - Lemmatizing the words

Spacy:

spaCy is an open-source natural language processing (NLP) library that's designed for various NLP tasks such as tokenization, part-of-speech tagging, named entity recognition, dependency parsing, and more. It's known for its efficiency and high-performance capabilities, making it a popular choice among researchers and developers working with text data.

1. Installation:

```
>>pip install spacy  
>>python -m spacy download en_core_web_md
```
2. Usage:

```
>> import spacy  
>>model = spacy.load('en_core_web_md')
```
3. Take clean data:

```
>> clean_document1="" ".join(clean_token)  
>>print(clean_document1)
```
4. Use the model:

```
>>result=model(clean_document1)
```
5. Print the label:

```
>>for entity in result.ents:  
    print(entity.entity.label_)
```

WordCloud:

The **wordcloud** library is a Python package that allows you to create visually appealing word clouds from text data. A word cloud is a graphical representation of word frequency in a given text, where the size of each word in the cloud is proportional to its frequency or importance. It's a popular tool for visualizing the most common words in a text document, and it's often used for data exploration, text analysis, and presentation purposes.

1. Importing:

```
>> from wordcloud import WordCloud
>> import matplotlib.pyplot as plt
```
2. Creating object:

```
>> wc=WordCloud(width=800,height=900,
                  background_color="black",
                  colormap="Oranges_r",
                  stopwords=["seamless","embark","Our","Your",
                            "ensure","Let","feature","u","Today","Cherish","We"],
                  min_font_size=10)
```
3. Used on NER Representation:

```
>>wc.generate(clean_document1)
>>plt.imshow(wc, interpolation='bilinear')
>>plt.axis("off")
>>plt.savefig("Img_1.jpg")
>>plt.show()
```